API

> La documentation **interactive et toujours à jour** est servie par l'application elle-même sur `http://localhost:3002/docs` (Swagger UI). Elle est générée depuis les modèles Pydantic de `app/schemas/api.py` : elle ne peut pas diverger du code. Ce document donne les contrats et les règles ; `/docs` donne les schémas exacts et permet d'essayer chaque route.

Base : `/api` · Format : JSON · Authentification : `Authorization: Bearer <jeton>`

---

1. Forme des réponses

Succès : l'objet demandé, sous une clé nommée.

```
{ "course": { "id": "…", "title": "…" } }
```

Échec : toujours la même forme, quelle que soit l'origine de l'erreur.

```
{
  "error": {
    "code": "FORBIDDEN",
    "message": "Ce cours ne vous appartient pas.",
    "details": {}
  }
}
```

Le front lit `error.code` et compose le message affiché depuis `client/src/lib/api.ts`. **Aucune page ne rédige son propre message** : c'est ce qui garantit que les cinq cas `LLM_*` s'affichent de la même façon partout, et qu'ils sont démontrables en soutenance.

Les erreurs levées par le cadriciel lui-même (404 sur une route inconnue, 405 sur une méthode non autorisée) sont réécrites dans cette forme. Il n'existe donc qu'un seul contrat d'erreur.

2. Codes d'erreur

Code	HTTP	Signification
`VALIDATION_ERROR`	400	entrée invalide ; `details` liste les champs
`UNAUTHORIZED`	401	jeton absent, invalide ou expiré
`FORBIDDEN`	403	la ressource ne vous appartient pas
`NOT_FOUND`	404	ressource ou route inexistante
`RATE_LIMITED`	429	*réservé, non émis actuellement*
`LLM_KEY_MISSING`	503	clé d'API absente ou refusée
`LLM_TIMEOUT`	504	le modèle n'a pas répondu dans le délai
`LLM_QUOTA_EXCEEDED`	429	quota du fournisseur atteint
`LLM_INVALID_OUTPUT`	502	sortie non conforme au schéma après relance
`LLM_UNAVAILABLE`	502	service indisponible, ou modèle inexistant
`INTERNAL`	500	erreur inattendue ; détails journalisés, jamais exposés

Les cinq codes `LLM_*` sont distincts à dessein : ce sont eux qui prouvent que l'exigence « gestion des erreurs d'appel API (timeout, quota, clé absente) » de l'énoncé est réellement traitée. Un message générique ne prouverait rien.

---

3. Authentification

`POST /api/auth/register` → 201

```
{ "email": "etudiant@esprit.tn", "password": "au moins 8 caracteres" }
```

Réponse : `{ "user": {"id","email"}, "token": "…" }`

L'adresse est validée (`EmailStr`) : les domaines réservés comme `.test` sont refusés. Le mot de passe est tronqué à 72 octets avant bcrypt — explicitement, parce que bcrypt le ferait silencieusement.

`POST /api/auth/login` → 200

Même corps, même réponse. **Le message d'échec est identique** que l'adresse soit inconnue ou le mot de passe faux : les distinguer reviendrait à publier la liste des comptes existants.

`GET /api/auth/me` → 200 · *authentifié*

`{ "user": {"id","email"} }`

Le jeton est un JWT HS256 valable 7 jours.

4. Cours

Toutes authentifiées. **Aucun appel au modèle dans cette section** : importer un cours, c'est enregistrer du texte.

`GET /api/courses`

```
{ "courses": [{
    "id": "…", "title": "…", "subject": "…",
    "charCount": 6229, "createdAt": "…",
    "_count": { "summaries": 1, "quizzes": 0, "messages": 4 }
}] }
```

Le contenu n'est pas renvoyé : inutile et volumineux.

`GET /api/courses/{id}`

`{ "course": { …, "content": "…" } }` — 403 si le cours appartient à autrui.

`POST /api/courses` → **201**

```
{ "title": "Chapitre 3", "subject": "Systemes", "content": "…" }
```

`content` doit faire **au moins 200 caractères** : en deçà, l'analyse n'a pas de sens. **Refusé** au-delà de `MAX_COURSE_CHARS`, avec le dépassement exact — et non tronqué : voir DECISIONS.md 016.

`POST /api/courses/import` → **201** · ***multipart/form-data***

Champs : `file`, `title` et `subject` facultatifs.

Formats acceptés : **PDF** à couche texte, **Word** `.docx`, **PowerPoint** `.pptx`. Le format est choisi sur l'**extension** du nom de fichier, pas sur le type MIME annoncé — voir DECISIONS.md 017.

Refusé si : l'extension n'est pas reconnue, le fichier dépasse **25 Mo**, le document est endommagé, ou l'extraction rend moins de 200 caractères — **cas du PDF scanné ou du document fait d'images**, signalé explicitement plutôt que traité en silence. Les formats d'avant 2007 (`.doc`, `.ppt`, `.odt`, `.odp`) sont refusés avec le remède dans le message.

Le format et la taille sont vérifiés **avant** la lecture des octets : un fichier de 200 Mo n'est jamais mis en tampon.

`DELETE /api/courses/{id}` → **204**

Supprime en cascade résumés, quiz, tentatives et messages. Les **examens blancs survivent** : ils ne pendent pas du cours, et leurs questions y sont recopiées.

4 bis. Examen blanc multi-cours

> **Aucune de ces routes n'appelle le modèle**, sauf `POST /api/exams` avec `completer: true`. Les questions viennent des quiz déjà générés.

`POST /api/exams` → 201

```
{ "courseIds": ["..."], "questionCount": 20,
  "durationMin": 20, "completer": false }
```

`questionCount` entre 5 et 40, au plus 10 cours. `durationMin` absent ⇒ une minute par question. `completer: true` génère un quiz pour un cours qui n'en a pas — **coûte un appel par cours**, et c'est la seule branche soumise au plafond de débit.

Rend l'examen **sans les bonnes réponses** : `questions[]` ne contient que `question` et `options`. Le champ `warnings[]` dit quels cours n'ont pas pu fournir leur part, et pourquoi.

Refusé si aucun cours choisi n'a de question disponible — le message nomme le remède.

`GET /api/exams` → liste · `GET /api/exams/{id}` → reprise

`remainingS` est **recalculé par le serveur** à chaque lecture. Le décompte du navigateur n'est qu'un affichage.

`POST /api/exams/{id}/submit`

```
{ "answers": [0, null, 2, 3] }
```

`null` pour une question laissée vide : sur un examen chronométré c'est le cas normal, pas une anomalie.

C'est le seul endroit où les bonnes réponses descendent. La réponse porte `correction[]` — chaque ligne citant son `courseId` et `courseTitle` — et `parCours[]`, trié **du cours le moins bien réussi au mieux réussi** : c'est l'ordre dans lequel reprendre ses révisions.

Une copie déjà rendue est refusée en 400 : sans cela, il suffirait de lire le corrigé puis de renvoyer le formulaire.

---

4 ter. Recherche et tuteur transversaux

`GET /api/search?q=…&limit=10`

Cherche dans **tous** les cours de l'étudiant. **N'appelle pas le modèle** : purement lexical, donc instantané, gratuit, et fonctionnel même sans clé d'API. Pas de plafond de débit — il n'y a rien à protéger.

Rend `passages[]` — `courseId`, `courseTitle`, `rang` / `total` du segment, `extrait` (fenêtre centrée sur le mot trouvé, pas le début du segment) et `score` BM25 — plus `terms[]`, les mots effectivement cherchés. Le navigateur surligne à partir de `terms` pour appliquer **exactement** les mêmes règles que le classement.

Les passages de score nul ne sont pas rendus : une liste vide est une information, une liste de bruit n'en est pas une.

Les termes sont élargis à leurs variantes singulier/pluriel côté requête seulement — voir DECISIONS.md 019.

`POST /api/ask`

```
{ "question": "Où ai-je vu l'exécution symbolique ?" }
```

Un seul appel au modèle, quel que soit le nombre de cours : les trois passages les plus proches sont choisis en local avant l'appel. Plafonné, comme les autres routes qui appellent.

Chaque source commence par le **titre de son cours entre crochets**, suivi d'un extrait mot pour mot. `courses[]` nomme les cours consultés, même quand la réponse n'est pas ancrée : savoir *où* le tuteur a cherché fait partie de la réponse.

`appeleLeModele` vaut **faux** quand aucun passage ne correspondait : la réponse « non couverte » est alors produite sans appel.

La réponse **n'est pas enregistrée** : un échange qui traverse plusieurs cours ne se rattache à aucun. Voir DECISIONS.md 019.

---

5. Module 1 — Résumé

`POST /api/courses/{id}/summary`

```
{ "level": "court" | "standard" | "detaille" }
```

```
{ "summary": {
    "id": "…", "level": "standard",
    "payload": {
      "abstract": "…", "keyPoints": ["…"],
      "keywords": ["…"], "warnings": []
    },
    "model": "gemini-3.5-flash-lite", "promptVersion": "v1",
    "latencyMs": 4114, "segmentCount": 1, "createdAt": "…"
} }
```

Mis en cache sur `(cours, niveau)`. Sur un cours long, `segmentCount > 1` et le résumé résulte d'une agrégation.

Si le cours a malgré tout dû être amputé, `warnings` contient un message explicite destiné à l'étudiant.

`GET /api/courses/{id}/summaries`

`{ "summaries": [ … ] }`, du plus récent au plus ancien.

---

6. Module 2 — Quiz

`POST /api/courses/{id}/quiz`

```
{ "questionCount": 5, "difficulty": "standard" }
```

`questionCount` entre 3 et 20.

```
{ "quiz": {
    "id": "…", "questionCount": 5, "segmentCount": 1,
    "questions": [{ "question": "…", "options": ["A","B","C","D"] }]
} }
```

> **`correctIndex`, `explanation` et `sourceQuote` ne figurent pas dans cette réponse.** Ils existent en base mais sont retirés avant l'envoi : sinon l'auto-évaluation n'aurait aucun sens, il suffirait de lire le JSON dans l'inspecteur réseau. C'est vérifié par un test.

`POST /api/quizzes/{id}/submit`

```
{ "answers": [0, 2, -1] }
```

`-1` signale une question laissée sans réponse ; elle est comptée fausse.

```
{
  "attempt": { "id": "…", "score": 2, "total": 3, "createdAt": "…" },
  "correction": [{
      "question": "…", "chosen": 0, "correctIndex": 0, "correct": true,
      "explanation": "…", "sourceQuote": "…"
  }]
}
```

La correction est faite **côté serveur**. C'est seulement ici que la bonne réponse, l'explication et la citation descendent au navigateur.

---

7. Module 3 — Tuteur

`GET /api/courses/{id}/messages`

`{ "messages": [ … ] }`, dans l'ordre chronologique.

`POST /api/courses/{id}/chat`

```
{ "question": "Quel outil mesure la couverture de code ?" }
```

Entre 3 et 1 000 caractères.

```
{ "message": {
    "id": "…", "role": "assistant",
    "content": "L'outil mentionne est pytest-cov…",
    "sources": ["pip install pytest pytest-cov"],
    "grounded": true,
    "latencyMs": 3733, "createdAt": "…"
} }
```

`grounded: false` signifie que les passages retrouvés ne permettaient pas de répondre. Le front l'affiche par une pastille « Hors du cours ». Les six derniers messages servent de contexte conversationnel.

---

8. Espace révision

Aucun appel au modèle dans cette section : tout vient de la table des tentatives, qui enregistrait déjà ces données sans que personne ne les affiche.

`GET /api/courses/{id}/attempts`

```
{ "attempts": [
    { "id": "…", "quizId": "…", "score": 4, "total": 5,
      "difficulty": "standard", "createdAt": "…" }
] }
```

De la plus récente à la plus ancienne.

`GET /api/courses/{id}/progress`

```
{
  "attemptCount": 4,
  "averageScore": 50,
  "bestScore": 100,
  "lastScore": 0,
  "trend": [100, 100, 0, 0],
  "weakQuestions": [
    { "question": "…", "wrongCount": 2, "sourceQuote": "…" }
  ]
}
```

Les scores sont en **pourcentage** : un quiz de trois questions et un quiz de quinze ne se comparent pas autrement.

`weakQuestions` est le cœur de la fonctionnalité — les questions ratées **au moins deux fois**, avec l'extrait du cours correspondant. Une question ratée une seule fois peut l'être par inattention ; à partir de deux, c'est une notion à revoir.

> Même règle que pour le quiz : **`correctIndex` et `explanation` ne figurent pas dans cette réponse.** Vérifié par un test.

9. Transparence

`GET /api/summaries/{id}/payload`

```
{
  "payload": "…la charge exacte envoyee au modele…",
  "tokenMap": ["EMAIL", "ETABLISSEMENT", "TEL"]
}
```

C'est la seule preuve vérifiable de l'anonymisation : affirmer que les données sont masquées ne vaut rien, le montrer se vérifie à l'œil nu.

`tokenMap` liste les types **réellement** remplacés dans ce cours, lus depuis la table des jetons enregistrée au moment de l'appel. Une liste vide signifie « aucune donnée personnelle détectée », et le dit. *(Une première version cherchait les jetons dans la charge envoyée — qui contient le prompt système, lequel cite `[NOM_1]`, `[EMAIL_1]` … en exemple. L'écran affichait donc toujours les mêmes quatre types. Corrigé le 12 octobre.)*

---

10. Service

`GET /api/health` · ***non authentifiée***

```
{ "status": "ok", "provider": "gemini",
  "model": "gemini-3.5-flash-lite", "cle_presente": true }
```

Aucun secret : seulement la **présence** de la clé.

---

11. Règles transversales

1. Toute entrée est validée par un modèle Pydantic. Rien n'est lu directement depuis le corps de la requête.
2. Toute route autre que `/health`, `/register` et `/login` exige un jeton.
3. La propriété est vérifiée **côté serveur** à chaque accès. Les gardes du front ne protègent que l'affichage.
4. CORS limité à `CLIENT_ORIGIN`.
5. Les messages d'erreur n'exposent jamais de détail interne : trace, requête SQL ou chemin de fichier restent dans les journaux.